Campus Incident Reporting App

Technical Report - Individual Assignment 2

Samuel Kabera Nshuti | s.kabera@alustudent.com

GitHub: https://github.com/KaberaSamuel/campus-incident-reporting-app

Demo video (7-10 min): **[PASTE VIDEO LINK HERE]**

1. Introduction

I built Campus Incident Reporting, an Android app made with Flutter. Students use it to report campus problems such as broken facilities, water shortages, electricity outages, damaged furniture, sanitation problems and safety concerns. I made it so that students and campus staff have one place to report an issue and follow it until it is resolved. Students can attach up to three photos, search and filter reports, and see the status of each report. Staff can review all reports and update their status.

The app uses Firebase Authentication, Cloud Firestore and Cloud Functions. Photos are stored in Cloudflare R2 (see Section 4.3). The app is meant for ALU users only, so it accepts only *@alustudent.com* and *@alueducation.com* emails.

2. Application architecture

I organised the code in feature folders (*auth, incidents, locations, notifications, profile, staff, student*) and split each feature into screens, models, providers and repositories. I did this so that the UI never talks to Firebase directly. A screen calls a provider or a repository, and only the repository knows about Firestore. This made it easy for me to change queries without touching the widgets.

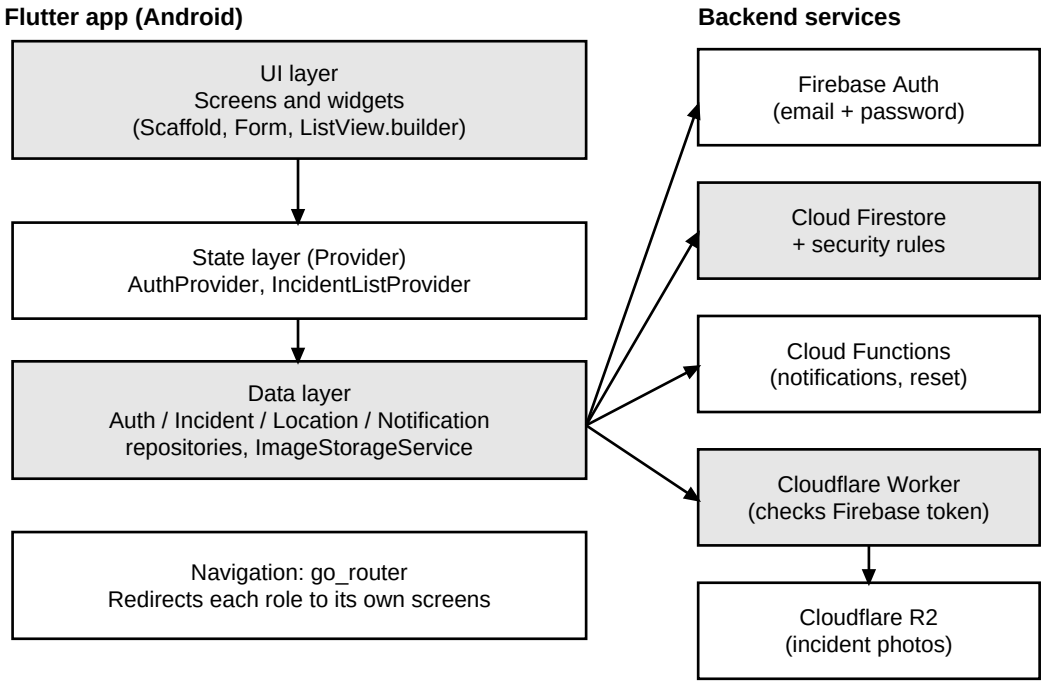

Figure 1. Architecture of the app and its backend services.

Navigation. I used *go_router* with two *ShellRoute*s, one for students and one for staff. Each shell shows a *NavigationBar* with that role's tabs (students: Home, Reports, Report, Inbox, Profile). The router has a *redirect* function and listens to the auth state, so a signed-out user is sent to login and a student can never open a staff screen.

3. Firebase data structure

I used four Firestore collections. I stored the location name and the reporter name inside each incident so that the list screen can show them without extra reads.

Collection	Main fields	Purpose
users/{uid}	email, fullName, role (student | staff), fcmTokens	Profile and role used for access control and push notifications
incidents/{id}	reporterId, reporterName, category, title, titleLowercase, description, locationId, locationName, imageUrls (max 3), status, staffNote, statusHistory[], createdAt, updatedAt	Every incident report
locations/{id}	name	Places a student can choose when reporting (managed by staff)
notifications/{id}	userId, type, title, body, incidentId, read, createdAt	In-app inbox, written by Cloud Functions

Categories and statuses. I used Dart enums for the category (facilities, water, electricity, furniture, sanitation, safety, other) and for the status (submitted, in_progress, resolved). Each enum has a *firestoreValue* so the stored text stays stable even if I change a label in the UI. Every status change adds an entry to *statusHistory*, which the detail screen shows as a timeline.

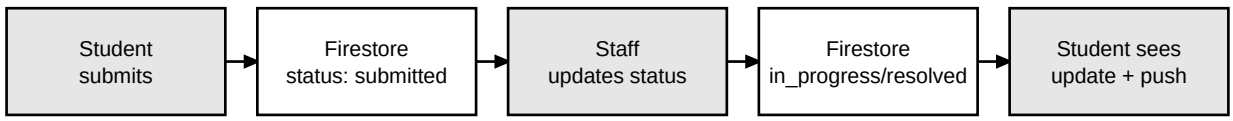

Figure 2. Life of an incident.

4. Technical decisions

4.1 State management: Provider

I chose **Provider** because it is simple and it is the approach we learned. In *main.dart* a *MultiProvider* exposes the repositories and services once, and screens read them with *context.read*. I used two *ChangeNotifier* classes:

- **AuthProvider** holds the signed-in user, their profile and role. Screens use *context.watch* on it, and *go_router* listens to it so a login or logout changes the screen right away.
- **IncidentListProvider** holds the list, the loading and error states, and the filters (search, category, status, location, my reports only). I did this so the list screen only draws the data and does not contain query logic. It also handles pagination: it loads 20 incidents at a time and calls *loadMore* when the user scrolls down.
- For one report that can change while it is open, I used a *StreamBuilder* on a Firestore *snapshots()* stream so the status updates live. I used *setState* only for small things like a form field or a loading flag inside one widget.

4.2 Key Flutter widgets

Widget	Where I used it and why
Scaffold, NavigationBar	Each role's shell has a bottom bar so the main tabs are always one tap away.
Form, TextFormField	Login, register and the report form validate input (empty title, short password, wrong email domain) before anything is sent.
DropdownButtonFormField	Choosing a category and a location when reporting.
FilterChip, search field	Filtering the report list by category and status, and searching by title.
ListView.builder, RefreshIndicator	Builds long incident lists lazily; pull down to refresh and scroll down to load more.
StreamBuilder	Live view of one incident and the inbox.
Image.network	Shows the incident photos from their R2 link.
Card, ListTile, SnackBar, AlertDialog	Reusable list items, quick feedback messages and confirmation before deleting.

I also made small reusable widgets in *core/widgets* (*StatusBadge*, *LoadingView*, *ErrorStateView*, *EmptyStateView*) so every screen shows loading, errors and empty lists the same way.

4.3 Photo storage: Cloudflare R2 instead of Firebase Storage

The assignment asks for photo uploads. Firebase Storage needs the paid Blaze plan, and I did not want to upgrade my plan as a student. Instead I used **Cloudflare R2**, which stores the photos without that cost. I want to be clear that I made this choice on purpose and that photos are still uploaded, stored and shown in the app.

An Android app cannot safely contain the R2 secret keys, so I wrote a small **Cloudflare Worker** (*worker/src/index.js*). The app sends the photo to the Worker together with the user's Firebase ID token. The Worker checks the token, then allows the upload only for ALU student accounts, with at most 3 photos per incident and 5 MB per photo. Delete is allowed for the reporter (only while the report is still *submitted*) or for staff. Before uploading, the app compresses each image with *flutter_image_compress* to save data. The Worker returns a public link, and I store that link in the incident's *imageUrls* field, so Firestore stays the source of truth for each report.

4.4 Authentication and email verification

I used Firebase Authentication with email and password. A user can only register with an ALU address. I added **email verification**: after sign-up Firebase sends a verification email, and the app only lets the user in once the email is verified, so the app can only be used with a real ALU mailbox. The role is chosen from the email domain (*@alustudent.com* = student, *@alueducation.com* = staff) and saved in *users/{uid}*. For the school project I also wrote a Cloud Function, *resetSchoolPassword*, so a password can be reset, and I added a change-password screen that asks for the current password first.

4.5 Push notifications

I used two Cloud Functions that listen to Firestore. When a student creates an incident, staff get a push notification (Firebase Cloud Messaging) and an inbox entry. When staff change a status, the reporter gets the same. Tapping a notification opens that incident. I did this so students do not need to keep checking the app.

5. User roles and access control

The app has two roles. Students submit and view reports. Staff review reports, change the status, add a staff note and manage locations. I enforce this in three places so that hiding a button is not the only protection:

- **Router:** *go_router* redirects each role to its own screens.

- **Firestore security rules:** only a student can create an incident (it must start as *submitted* and belong to the student); only staff can update an incident, and only the fields *status*, *staffNote*, *updatedAt*, *statusHistory*; a student can delete their own report only while it is *submitted*; only staff can edit locations; a user can read only their own profile and notifications.
- **Cloudflare Worker:** repeats the same upload and delete rules for photos.

Staff account used for the demo. A real staff account must use a real *alueducation.com* email. I am a student, so I do not have one. To show the staff role in my demo, I created the account *staff@alueducation.com*. It only passes the domain check and receives the staff role, and it is used for demonstration only. In a real deployment, staff accounts would be created from real ALU staff mailboxes, ideally by an administrator.

6. UI/UX decisions

I kept the design simple on purpose so that a student can report an issue in a few taps.

- The report form is one screen: category, title, description, location, optional photos, then Submit. Required fields are validated and the Submit button shows a progress indicator while uploading.
- Status is shown with a *StatusBadge* (Submitted, In progress, Resolved) on every list item and on the detail screen, with a timeline so students see when each change happened.
- Every list has a loading state, an error state with a retry, and a friendly empty state.
- Students can see all campus reports, find their own with a *You* tag or the *My reports* filter, and search and filter to avoid sending duplicates.
- The staff dashboard shows counts per status, so staff see what needs attention first.
- A student can submit only when staff have added at least one location, so reports always have a place.

[Insert screenshot: Login / register screen]

Figure 3. Authentication screens.

[Insert screenshot: Student home and report form]

Figure 4. Student: home screen and submitting an incident with a photo.

[Insert screenshot: Reports list with search/filters, and incident detail with timeline]

Figure 5. Browsing, filtering and viewing a report.

[Insert screenshot: Staff dashboard and status update]

Figure 6. Staff: dashboard and updating a status with a note.

[Insert screenshot: Firebase Console: users and incidents collections]

Figure 7. Firestore records created by the app.

7. Testing strategy

I used a mix of automated and manual testing:

- **Automated:** a unit test checks that an *Incident* is converted to Firestore data correctly (for example *titleLowercase* and *status*), a widget test checks that the app theme builds, and the Worker has a smoke test. I run them with *flutter test*.

- **Manual checklist** (in the README), run on an Android emulator and a device: register and log in with both email domains; reset and change password; log out; submit a report with 0 to 3 photos; search and filter; delete my own report only while it is *submitted*; staff status update and note; manage locations (delete fails when a location is used); push notification and inbox; tapping a notification opens the incident.
- **Security testing:** I tried actions a normal user should not be able to do, such as a student changing a status, and checked that the Firestore rules rejected them.

8. Challenges and lessons learned

- **Photo storage without paying.** Firebase Storage needed an upgrade, so I learned how to use Cloudflare R2 and to put a Worker in front of it so no secret key ships in the APK.
- **Search in Firestore.** Firestore has no "contains" search. I stored *titleLowercase* and used a prefix query on it, which needs a matching index. The trade-off is that search matches the start of the title only.
- **Security rules.** My first rules were too open. I learned to write rules that check the role and the exact fields that may change, instead of trusting the app.
- **Profile document missing.** An account sometimes existed without a profile (when the rules were not yet deployed). I added a repair step in *AuthRepository.fetchUserProfile* that creates it.
- **Firestore database ID.** The app failed to start when the database name in the code did not match the one in the console, so I put the ID in one constant.
- **Lesson:** keeping UI, state and data in separate layers made all of these fixes small and local.

9. Limitations and future work

- Search matches the beginning of a title only; a search service could fix this.
- Staff accounts are decided by email domain; a production app should assign roles from an admin tool (for example Firebase custom claims).
- The R2 photo links are public, so anyone with a link can view a photo.
- More automated tests (providers and repositories) would be useful.

References

[1] Google, "Flutter documentation," 2026. [Online]. Available: https://docs.flutter.dev

[2] Google, "Firebase Authentication," 2026. [Online]. Available: https://firebase.google.com/docs/auth

[3] Google, "Cloud Firestore and security rules," 2026. [Online]. Available: https://firebase.google.com/docs/firestore

[4] Google, "Cloud Functions for Firebase," 2026. [Online]. Available: https://firebase.google.com/docs/functions

[5] Google, "Firebase Cloud Messaging," 2026. [Online]. Available: https://firebase.google.com/docs/cloud-messaging

[6] Cloudflare, "R2 object storage and Workers," 2026. [Online]. Available: https://developers.cloudflare.com/r2

[7] R. Rousselet, "provider package," pub.dev, 2026. [Online]. Available: https://pub.dev/packages/provider

[8] Flutter team, "go_router package," pub.dev, 2026. [Online]. Available: https://pub.dev/packages/go_router